

26

ESTIMATIVAS DE PROJETO DE SOFTWARE

CONCEITOS-CHAVE

escopo do software	606
equação do software	621
estimativa	605
ágil	622
baseada em FP	612
baseadas em problemas	611
baseadas em processo	614
casos de uso	616
modelos empíricos	618
projetos orientados a objeto	622
reconciliação	617
WebApps	623
planejamento do projeto	606
viabilidade	606

O gerenciamento de projeto de software começa com uma série de atividades chamadas coletivamente de *planejamento de projeto*. Antes de iniciar o projeto, a equipe de software deverá fazer uma estimativa do trabalho, os recursos que serão necessários e o tempo necessário para a sua conclusão. Uma vez executadas essas atividades, a equipe de projeto deverá estabelecer um cronograma que defina as tarefas de engenharia de software e as metas, deverá identificar os responsáveis pela execução de cada tarefa e especificar as dependências entre tarefas que podem ter forte influência no progresso do trabalho.

Em um excelente guia de “sobrevivência de projeto de software”, Steve McConnell [McC98] apresenta uma visão do planejamento de projeto na prática:

Muitos profissionais técnicos gostariam de ir direto ao trabalho em vez de passar o tempo planejando. Muitos gerentes técnicos não possuem treinamento suficiente em gerenciamento para se sentirem confiantes de que o seu planejamento melhorará os resultados de um projeto. Como nenhuma das partes quer realizar planejamento, a tarefa em geral acaba não sendo cumprida.

Mas não planejar é uma das falhas mais críticas que um projeto pode ter... O planejamento eficaz é necessário para resolver problemas precoces [no início do projeto] com baixo custo, em vez de problemas tardios [que surgem mais tarde no projeto] com alto custo. Os projetos consomem em média 80% do tempo em retrabalho – corrigindo erros que foram feitos no início do projeto.

PANORAMA

O que é? Uma necessidade real de software foi estabelecida; as pessoas interessadas estão envolvidas, os engenheiros de software estão prontos para começar, e o projeto está para ser iniciado. E como você procede? O planejamento de projeto de software abrange cinco atividades importantes – estimativa, cronograma, análise de risco, planejamento do gerenciamento de qualidade e planejamento para o gerenciamento de alterações. Neste capítulo, consideraremos somente as estimativas – a tentativa para determinar quanto dinheiro, esforço, recursos e tempo serão necessários para criar um sistema ou artefato baseado em software específico.

Quem realiza? Os gerentes de projeto de software – usando informações recebidas dos interessados no projeto e dados das métricas de software coletados de projetos anteriores.

Por que é importante? Você construiria uma casa sem saber quanto está disposto a gastar, as tarefas que precisam ser executadas e os prazos para executar o trabalho? É claro que não, e como a maioria dos sistemas e produtos baseados em computadores custa consideravelmente mais para ser construídos do que a construção de

uma grande casa, parece bem razoável desenvolver uma estimativa antes de começar a criar o software.

Quais são as etapas envolvidas? As estimativas começam com uma descrição do escopo do problema. O problema é então decomposto em uma série de questões menores, e cada um desses problemas é estimado por meio de dados históricos e da experiência como guia. A complexidade e os riscos do problema são considerados antes que a estimativa final seja feita.

Qual é o artefato? É gerada uma tabela simples que descreve as tarefas a ser executadas, as funções a ser implementadas, e o custo, esforço e tempo envolvidos para cada atividade.

Como garantir que o trabalho foi executado corretamente? Isso é difícil, porque não se pode realmente saber até que o projeto seja finalizado. No entanto, se você tiver experiência e seguir uma abordagem sistemática, se gerar estimativas usando dados históricos confiáveis, se criar dados de estimativa usando pelo menos dois métodos diferentes, se estabelecer um cronograma realístico e se adaptá-lo continuamente à medida que o projeto avança, pode-se ter certeza de que foi feita a melhor escolha.

McConnell alega que toda equipe pode arranjar espaço para planejar (e adaptar o plano durante o projeto) simplesmente reservando uma pequena porcentagem do tempo que teria sido gasto no retrabalho porque o planejamento não foi feito.

26.1 OBSERVAÇÕES E ESTIMATIVAS

O planejamento requer que você assuma um comprometimento inicial, mesmo que este venha a demonstrar mais tarde que estava errado. Sempre que forem feitas estimativas, deve-se olhar o futuro e aceitar certo grau de incerteza. Segundo Frederick Brooks [Bro95]:

...Nossas técnicas de estimativa são muito mal desenvolvidas. E, pior, refletem uma suposição bastante falsa, não declarada, de que tudo sairá muito bem... Porque não temos certeza sobre nossas estimativas, gerentes de software muitas vezes não possuem a firmeza necessária para fazer as pessoas esperarem por um bom produto.

Embora estimar seja muito mais arte do que ciência, não precisa ser conduzida de maneira aleatória. Existem técnicas úteis para estimar tempo e esforço. As métricas de projeto e processo podem proporcionar perspectivas históricas e valiosas informações para gerar estimativas quantitativas. A experiência (de todos os envolvidos) pode ajudar imensamente à medida que as estimativas são desenvolvidas e revisadas. Por serem a base para todas as outras ações do planejamento de projeto, e pelo fato de o planejamento de projeto fornecer a direção para uma engenharia de software bem-sucedida, seria uma péssima ideia iniciar sem as estimativas.

As estimativas de recursos, custos e cronograma para um trabalho de engenharia de software requerem experiência, acesso a boas informações históricas (métricas), e a coragem de se comprometer com as previsões quantitativas quando tudo o que existe são apenas informações qualitativas. A estimativa traz um risco inerente,¹ e esse risco leva à incerteza.

A *complexidade do projeto* tem um forte efeito sobre a incerteza inerente ao planejamento. No entanto, é uma medida relativa afetada pela familiaridade com esforços passados. Alguém que pela primeira vez desenvolve uma sofisticada aplicação para comércio eletrônico pode considerá-la excessivamente complexa. No entanto, uma equipe de engenharia para Web desenvolvendo sua décima aplicação WebApp para comércio eletrônico consideraria isso um trabalho comum. Já foram propostas várias medidas de complexidade quantitativa de software [Zus97]. Estas são aplicadas em nível de projeto ou código e são, portanto, difíceis de usar durante o planejamento de software (antes do projeto e código). No entanto, outras avaliações mais subjetivas de complexidade (por exemplo, fatores de ajuste de complexidade de pontos de função descritos no Capítulo 23) podem ser estabelecidas no início do processo de planejamento.

O *tamanho do projeto* é outro fator importante que pode afetar a precisão e a eficácia das estimativas. À medida que o tamanho aumenta, a interdependência entre os vários elementos do software cresce rapidamente.² A decomposição do problema, uma abordagem importante para a estimativa, torna-se mais difícil, porque o refinamento dos elementos envolvidos pode ainda ser considerável. Parafraseando a lei de Murphy: “O que pode sair errado, sairá errado” – e se houver mais itens que podem falhar, mais itens falharão.

O *grau de incerteza estrutural* também tem um efeito sobre o risco das estimativas. Nesse contexto, a estrutura refere-se ao grau segundo o qual os requisitos foram solidificados, a facilidade com a qual as funções podem ser separadas e a natureza hierárquica das informações a ser processadas.

A disponibilidade de informações históricas tem uma forte influência sobre o risco das estimativas. Utilizar procedimentos que funcionaram pode melhorar áreas problemáticas. Quando

“Boas abordagens de estimativa e dados históricos sólidos são a maior esperança de que a realidade vencerá as demandas impossíveis.”

Caper Jones

PONTO-CHAVE

A complexidade do projeto, tamanho e grau de incerteza estrutural afetam a confiabilidade das estimativas.

¹ Técnicas sistemáticas para análise de riscos são apresentadas no Capítulo 28.

² O tamanho muitas vezes aumenta devido ao “deslizamento do escopo” que ocorre quando os requisitos do problema mudam. O aumento no tamanho do projeto pode ter um impacto geométrico sobre o custo e o cronograma do projeto (Michael Mah, comunicação pessoal).

“É característica de uma mente instruída satisfazer-se com o grau de precisão que a natureza do assunto permite, e não procurar exatidão quando apenas uma aproximação da verdade é possível.”

Aristóteles

há disponíveis métricas de software abrangentes (Capítulo 25) para projetos passados, as estimativas podem ser feitas com maior segurança, podem ser estabelecidos os cronogramas para evitar dificuldades passadas, e o risco em geral é reduzido.

O risco das estimativas é medido pelo grau de incerteza nas estimativas quantitativas estabelecidas para os recursos, custo e cronograma. Se o escopo do projeto é mal entendido ou se os requisitos do projeto sofrem alterações, a incerteza e o risco das estimativas tornam-se perigosamente altos. Como planejador, você e o cliente devem reconhecer que variabilidade nos requisitos de software significa instabilidade nos custos e no cronograma.

No entanto, você não deve se tornar obsessivo em relação às estimativas. Abordagens modernas de engenharia de software (por exemplo, modelos incrementais de processo) assumem uma visão iterativa do desenvolvimento. Em tais abordagens, é possível – embora nem sempre politicamente aceitável – voltar à estimativa (conforme mais informações são conhecidas) e revisá-la quando o cliente fizer alterações nos requisitos.

26.2 O PROCESSO DE PLANEJAMENTO DO PROJETO



Quanto mais você sabe, melhor você estima. Portanto, atualize suas estimativas à medida que o projeto avança.

O objetivo do planejamento de software é proporcionar uma estrutura que permite ao gerente fazer estimativas razoáveis dos recursos, custo e cronograma. Além disso, as estimativas devem tentar definir cenários de melhor e pior caso para que os resultados do projeto possam ser delineados. Embora haja um grau inerente de incerteza, a equipe de software participa de um plano estabelecido como consequência dessas tarefas. O plano deve ser adaptado e atualizado à medida que o projeto avança. Nas próximas seções, discutiremos cada uma das ações associadas a planejamento de projeto de software.

CONJUNTO DE TAREFAS



Conjunto de tarefas para planejamento de projeto

1. Estabeleça o escopo do projeto.
2. Determine a viabilidade.
3. Analise os riscos (Capítulo 28).
4. Defina os recursos necessários.
 - a. Determine os recursos humanos necessários.
 - b. Defina recursos de software reutilizáveis.
 - c. Identifique recursos ambientais.
5. Estime o custo e a mão de obra.
 - a. Decomponha o problema.
 - b. Desenvolva duas ou mais estimativas usando tamanho, pontos de função, tarefas de processo ou casos de uso.
 - c. Reconcilie as estimativas.
6. Desenvolva um cronograma de projeto (Capítulo 27).
 - a. Estabeleça um conjunto significativo de tarefas.
 - b. Defina uma rede de tarefas.
 - c. Use ferramentas de cronograma para desenvolver um diagrama de tempos.
 - d. Defina mecanismos para acompanhamento do cronograma.

26.3 ESCOPO E VIABILIDADE DO SOFTWARE

O *escopo do software* descreve as funções e características que devem ser fornecidas aos usuários finais; os dados que entram e saem; o “conteúdo” que é apresentado aos usuários como consequência do uso do software; e o desempenho, restrições, interfaces e confiabilidade que *limitam* o sistema. O escopo é definido por meio de uma das duas técnicas:

1. Uma descrição narrativa do escopo do software é desenvolvida após comunicação com todos os interessados.
2. É desenvolvida uma série³ de casos de uso pelos usuários finais.

³ Casos de uso foram discutidos em detalhe na Parte 2 deste livro. Um caso de uso é uma descrição baseada em cenário da interação do usuário com o software sob o ponto de vista do usuário.



A viabilidade do projeto é importante, mas uma consideração das necessidades do negócio é ainda mais importante. Não é uma boa ideia criar um sistema ou produto de alta tecnologia que ninguém quer.

Funções descritas na definição do escopo (ou nos casos de uso) são avaliadas e em algumas situações refinadas para fornecer mais detalhes antes de iniciar as estimativas. Como as estimativas de custo e cronograma são ambas funcionalmente orientadas, muitas vezes é aconselhável certo grau de decomposição. As considerações de desempenho abrangem requisitos de processamento e tempo de resposta. As restrições identificam limites colocados no software por hardware externo, memória disponível ou outros sistemas existentes.

Uma vez identificado o escopo (com a participação do cliente), é razoável perguntar: “Podemos criar software que atenda a esse escopo? O projeto é viável?”. Muitas vezes, os engenheiros de software passam rapidamente por essas questões (ou são empurrados por gerentes impacientes ou outras pessoas interessadas), somente para se envolverem em um projeto que já está condenado desde o início. Putnam e Myers [Put97a] tratam do assunto quando escrevem:

Nem tudo o que é imaginável é viável, nem mesmo no software, transiente como pode parecer para quem está de fora. Por outro lado, a viabilidade do software tem quatro dimensões sólidas: *Tecnologia* – O projeto é tecnicamente viável? Ele está incluído no estado da arte? Os erros podem ser reduzidos até um nível que corresponda às necessidades da aplicação? *Finanças* – Ele é financeiramente viável? O desenvolvimento pode ser completado a um custo que a organização de software, seus clientes ou o mercado podem pagar? *Tempo* – O tempo que o projeto leva para atingir o mercado vence a concorrência? *Recursos* – A organização tem os recursos necessários para ser bem-sucedida?

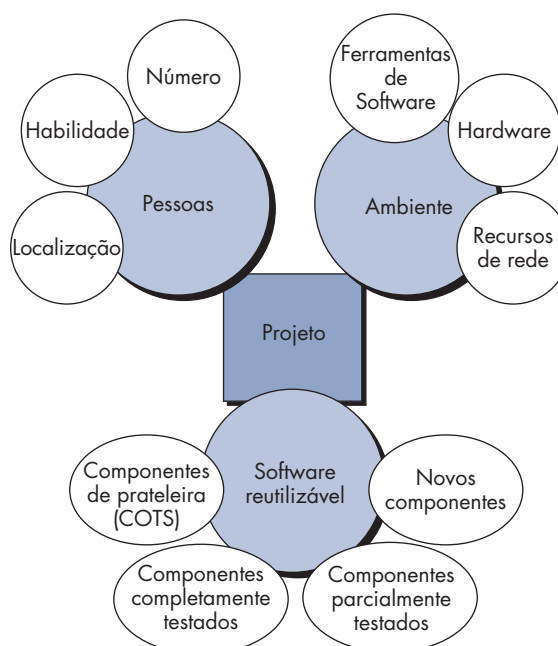
Putnam e Myers sugerem corretamente que só escopo não é suficiente. Uma vez entendido o escopo, deve-se trabalhar para determinar se ele pode ser feito segundo as dimensões que acabamos de descrever. Essa é uma parte crucial do processo de estimativas, que muitas vezes passa despercebida.

26.4 RECURSOS

A segunda tarefa do planejamento é a estimativa dos recursos necessários para executar o trabalho de desenvolvimento de software. A Figura 26.1 mostra as três principais categorias de recursos de engenharia de software – pessoas, componentes de software reutilizáveis e o ambiente de desenvolvimento (hardware e ferramentas de software). Cada recurso é especificado

FIGURA 26.1

Recursos de projeto



com quatro características: descrição do recurso, uma definição da disponibilidade, instante em que o recurso será necessário e tempo durante o qual o recurso será requerido. As duas últimas podem ser vistas como uma *janela de tempo*. A disponibilidade do recurso para uma janela de tempo especificada deve ser estabelecida o mais cedo possível.

26.4.1 Recursos humanos

O planejador começa avaliando o escopo do software e selecionando as habilidades necessárias para completar o desenvolvimento. São especificadas a posição organizacional (por exemplo, gerente, engenheiro de software sênior) e especialização (por exemplo, telecomunicações, base de dados, cliente-servidor). Para projetos relativamente pequenos (poucas pessoas-mês), um único profissional pode executar todas as tarefas de engenharia de software, consultando os especialistas quando necessário. Para projetos maiores, a equipe pode estar geograficamente dispersa em localizações diferentes. Portanto, é especificada a localização de cada recurso humano.

O número de pessoas necessárias para um projeto de software pode ser determinado somente após uma estimativa do esforço de desenvolvimento (por exemplo, pessoas-mês). As técnicas para estimar o esforço são discutidas mais adiante neste capítulo.

26.4.2 Recursos de software reutilizáveis

A engenharia de software baseada em componente (*component-based software engineering* – CBSE)⁴ enfatiza a reusabilidade – isto é, a criação e reutilização de blocos básicos de software. Esses blocos básicos, chamados também de *componentes*, devem ser catalogados para facilitar a referência, padronizados para facilitar a aplicação e validados para facilitar a integração. Ben-natan [Ben00] sugere quatro categorias de recursos de software que deverão ser consideradas conforme progride o planejamento:



Nunca se esqueça de que integrar uma variedade de componentes reutilizáveis pode ser um desafio significativo. Pior ainda, o problema da integração reaparece à medida que vários componentes são atualizados.

Componentes de prateleira. Software existente que pode ser adquirido de terceiros ou de projetos anteriores. Componentes de prateleira (*commercial off-the-shelf* – COTS) são comprados de terceiros, prontos para uso no projeto atual e que foram totalmente validados.

Componentes totalmente testados. Especificações, projetos, código ou dados de teste desenvolvidos para projetos anteriores, similares ao software a ser criado para o projeto atual. Os membros da equipe de software atual têm experiência plena na área de aplicação representada por esses componentes. Portanto, as modificações necessárias em componentes totalmente testados apresentarão um risco relativamente baixo.

Componentes parcialmente testados. Especificações, projetos, código ou dados de teste desenvolvidos para projetos anteriores que estão relacionados com o software a ser criado para o projeto atual, mas que necessitará de uma grande modificação. Os membros da equipe de software atual têm uma experiência limitada na área de aplicações representada por esses componentes. Portanto, as modificações necessárias para componentes parcialmente testados terão um grau razoável de risco.

Novos componentes. Componentes de software que devem ser criados pela equipe especificamente para as necessidades do projeto atual.

Ironicamente, os componentes de software reutilizáveis são muitas vezes negligenciados durante o planejamento, só para se tornarem uma preocupação suprema mais adiante no processo de software. É aconselhável especificar requisitos de recursos de software o quanto antes. Dessa maneira, pode ser feita a avaliação técnica das alternativas e a aquisição realizada a tempo.

26.4.3 Recursos de ambiente

O ambiente que suporta um projeto de software, muitas vezes chamado de *ambiente de engenharia de software* (*software engineering environment* – SEE), incorpora hardware e soft-

⁴ CBSE é tratado no Capítulo 10.

ware. O hardware fornece uma plataforma que suporta as ferramentas (software) necessárias para produzir os artefatos resultantes de uma boa prática de engenharia de software.⁵ Devido a muitas organizações de software terem múltiplas localizações que requerem acesso ao SEE, deve-se prescrever a janela de tempo necessária para hardware e software e verificar se esses recursos estarão disponíveis.

Quando um sistema baseado em computador (incorporando hardware e software especializado) deve ser desenvolvido, a equipe pode necessitar de acesso a elementos de hardware que estão sendo desenvolvidos por outras equipes de engenharia. Por exemplo, o software para um dispositivo robótico usado em uma célula de manufatura pode necessitar de um robô específico (por exemplo, um soldador robótico) como parte da etapa de teste de validação; um projeto avançado para layout de página pode necessitar de um sistema digital de impressão de alta velocidade em algum instante durante o desenvolvimento. Cada elemento de hardware precisa ser especificado como parte do planejamento.

26.5 ESTIMATIVA DO PROJETO DE SOFTWARE

“Em uma época de terceirização e concorrência cada vez maior, a habilidade para estimar com mais precisão... emergiu como um fator crítico de sucesso para muitos grupos de TI.”

Rob Thomsett

As estimativas de custo e esforço de software nunca serão uma ciência exata. Muitas variáveis – fatores humanos, técnicos, ambientais e políticos – podem afetar o custo final do software e o esforço necessário para desenvolvê-lo. No entanto, as estimativas de projeto de software podem ser transformadas de algo sobrenatural para uma série de etapas sistemáticas que proporcionam estimativas com um risco aceitável. Para conseguir estimativas confiáveis de custo e esforço, surge uma série de opções:

1. Adie a estimativa no decorrer do projeto (obviamente, podemos conseguir uma estimativa com precisão de 100% depois que o projeto estiver completo!).
2. Fundamente suas estimativas em projetos similares que já foram completados.
3. Use técnicas de decomposição relativamente simples para gerar estimativas de custo de projeto e esforço.
4. Use um ou mais modelos empíricos para estimativa de custo e esforço do software.

Infelizmente, a primeira opção, embora atrativa, não é prática. As estimativas de custo devem ser feitas no início. No entanto, é preciso reconhecer que quanto mais esperar, mais informação você terá da realidade, e menor será a probabilidade de que cometa erros graves nas suas estimativas.

A segunda opção pode funcionar razoavelmente bem, se o projeto atual for muito similar a trabalhos anteriores e outras influências de projeto (por exemplo, o cliente, as condições comerciais, o ambiente de engenharia de software, prazos finais) forem quase equivalentes. Infelizmente, a experiência não tem sido sempre um bom indicador de resultados futuros.

As demais opções são abordagens viáveis para estimativa de projeto de software. Idealmente, as técnicas mencionadas para cada opção deverão ser aplicadas em paralelo; cada uma sendo usada para verificar a outra. As técnicas de decomposição assumem uma abordagem do tipo “dividir para conquistar” para a estimativa do projeto de software. Decompondo um projeto em suas funções principais e atividades de engenharia de software relacionadas, as estimativas de custo e esforço podem ser feitas passo a passo. Podem ser usados modelos empíricos de estimativa para complementar as técnicas de decomposição e oferecer uma abordagem de estimativa potencialmente valiosa por si mesma. Um modelo é baseado na experiência (dados históricos) e toma a seguinte forma:

$$d = f(v_i)$$

“É muito difícil fazer uma defesa vigorosa, plausível e arriscada de uma estimativa não originada de um método quantitativo, suportada por dados escassos e certificada principalmente pelas pressões dos gerentes.”

Fred Brooks

5 Outro hardware – o *ambiente-alvo* – é o computador no qual o software será executado quando liberado para o cliente final.

em que d é um entre uma série de valores estimados (por exemplo, esforço, custo, duração do projeto) e v_i são parâmetros independentes selecionados (por exemplo, LOC estimado ou FP).

Ferramentas automáticas de estimativa implementam uma ou mais técnicas de decomposição ou modelos empíricos e proporcionam uma opção atrativa para estimativas. Em tais sistemas, são descritos as características da organização de desenvolvimento (por exemplo, experiência, ambiente) e o software a ser desenvolvido. Com base nesses dados são estimados o custo e o esforço.

Cada uma dessas opções viáveis de estimativa de custo de software é apenas tão boa quanto os dados históricos usados para compor a estimativa. Se não existem dados históricos, os custos se baseiam em uma fundação muito precária. No Capítulo 25, examinamos as características de algumas das métricas de software que proporcionam a base para dados de estimativa histórica.

26.6 TÉCNICAS DE DECOMPOSIÇÃO

A estimativa de projeto de software é uma forma de solução de problema e, na maioria dos casos, o problema a ser resolvido (desenvolver uma estimativa de custo e esforço para um projeto de software) é muito complexo para ser considerado em uma única parte. Por essa razão, você deve decompor o problema, redefinindo-o como uma série de problemas menores (e, talvez, mais controláveis).

No Capítulo 24, a abordagem da decomposição foi discutida com base em dois pontos de vista: decomposição do problema e decomposição do processo. As estimativas usam uma ou ambas as formas de particionamento. Mas antes de fazer uma estimativa, entenda o escopo do software a ser criado e gere uma estimativa de seu “tamanho”.

26.6.1 Dimensionamento do software

A precisão de uma estimativa de projeto de software é baseada em vários itens:

- (1) o grau com que você estimou adequadamente o tamanho do produto a ser criado;
- (2) a habilidade para traduzir a estimativa de esforço humano, tempo de trabalho e recursos financeiros (uma função da disponibilidade de métricas de software confiáveis de projetos anteriores);
- (3) o grau com que o plano do projeto reflete as habilidades da equipe de software; e
- (4) a estabilidade dos requisitos do produto e o ambiente que suporta o trabalho de engenharia de software.

Nesta seção, consideraremos o problema do *dimensionamento do software*. Como a estimativa do projeto é apenas tão boa quanto a estimativa do trabalho a ser realizado, o dimensionamento representa seu primeiro grande desafio como planejador. No contexto do planejamento de projeto, tamanho refere-se a um resultado quantificável do projeto de software. Se for adotada uma abordagem direta, o tamanho pode ser medido em linhas de código (LOC). Se for escolhida uma abordagem indireta, o tamanho é representado como pontos de função (FP).

Putnam e Myers [Put92] sugerem quatro abordagens diferentes para o problema do dimensionamento:

- *Dimensionamento de “lógica nebulosa” (fuzzy logic)*. Essa abordagem usa as técnicas de raciocínio aproximado, a pedra fundamental da *lógica nebulosa*. Para utilizá-la, o planejador deve identificar o tipo de aplicação, estabelecer sua magnitude em uma escala qualitativa e então refinar a magnitude dentro do intervalo original.
- *Dimensionamento de pontos de função*. O planejador desenvolve estimativas das características do domínio da informação discutido no Capítulo 23.
- *Dimensionamento de componentes-padrão*. O software é composto por uma série de diferentes “componentes-padrão” que são genéricos em relação a determinada área de aplicação. Por exemplo, os componentes-padrão para um sistema de informações são

PONTO-CHAVE

O “tamanho” do software a ser criado pode ser estimado usando-se uma medida direta, LOC, ou uma medida indireta, FP.



Como dimensionamos o software que estamos planejando criar?

subsistemas, módulos, telas, relatórios, programas interativos, programas em lote, arquivos, LOC e instruções em nível de objeto. O planejador do projeto estima o número de ocorrências de cada componente-padrão e usa dados históricos de projeto para estimar o tamanho de cada um deles.

- *Dimensionamento de alteração.* Essa abordagem é empregada quando um projeto abrange o uso de software existente que deve ser modificado de alguma forma como parte do projeto. O planejador estima o número e tipo (por exemplo, reutilização, adição de código, mudança de código, exclusão de código) de modificações que devem ser feitas.

Putnam e Myers sugerem que os resultados de cada uma dessas abordagens de dimensionamento sejam combinados estatisticamente para criar uma estimativa de *três pontos* ou *valor esperado*. Isso é obtido desenvolvendo-se valores otimistas (baixo), mais prováveis e pessimistas (alto) para o tamanho e combinando-os por meio da Equação (26.1), descrita na Seção 26.6.2.

26.6.2 Estimativa baseada em problema

No Capítulo 25, linhas de código e pontos de função foram descritos como medidas por meio das quais podem ser calculadas as métricas de produtividade. Dados de LOC e FP são usados de duas maneiras durante a estimativa do projeto de software: (1) como variáveis de estimativa para “dimensionar” cada elemento do software e (2) como métricas de referência coletadas de projetos anteriores e utilizadas em conjunto com variáveis de estimativa para desenvolver projeções de custo e esforço.

Estimativas LOC e FP são técnicas distintas. No entanto, ambas têm muitas características em comum. Inicia-se com uma definição delimitada do escopo do software e daí tenta-se decompor a definição em funções de problemas que podem ser estimados individualmente. LOC ou FP (a variável de estimativa) é então estimada para cada função. Como alternativa, pode-se escolher um outro componente para dimensionamento como classes ou objetos, alterações ou processos de negócio afetados.

Métricas de produtividade de referência (por exemplo, LOC/pm ou FP/pm⁶) são aplicadas à variável apropriada de estimativa e, assim, se obtém o custo ou esforço para a função. As estimativas de função combinam-se para produzir uma estimativa geral para todo o projeto.

É importante observar, porém, que muitas vezes há uma dispersão substancial em métricas de produtividade para uma organização, não sendo aconselhável o uso de uma única métrica de produtividade de referência. Em geral, médias LOC/pm ou FP/pm deverão ser computadas por domínio de projeto. Os projetos deverão ser agrupados por tamanho de equipe, por área de aplicação, complexidade e outros parâmetros relevantes. Deverão ser calculadas as médias locais de domínio. Quando é estimado um novo projeto, esse deverá primeiro ser alocado a um domínio e, depois, à média de domínio apropriada para produtividade anterior deverá ser usada para gerar a estimativa.

As técnicas de estimativa LOC e FP diferem em nível de detalhe requerido para a decomposição e no alvo do particionamento. Quando é usada a LOC como variável de estimativa, a decomposição é absolutamente essencial e muitas vezes é adotada com níveis consideráveis de detalhes. Quanto maior o grau de particionamento, maior a probabilidade de serem desenvolvidas estimativas LOC razoavelmente precisas.

Para estimativas FP, a decomposição funciona de forma diferente. Em vez de focalizar-se na função, é estimada cada uma das características do domínio de informação – entradas, saídas, arquivos de dados, consultas e interfaces externas –, bem como os 14 valores de ajuste de complexidade discutidos no Capítulo 23. As estimativas resultantes podem então ser usadas para derivar um valor de FP que pode ser relacionado a dados anteriores e usado para gerar uma estimativa.

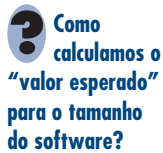
Independentemente da variável de estimativa utilizada, deve-se começar estimando um intervalo de valores para cada valor de função ou domínio de informação. Usando dados histó-

 **O que as estimativas baseadas em LOC e FP têm em comum?**



Ao coletar métricas de produtividade para projetos, estabeleça uma classificação de tipos de projeto. Isso permitirá o cálculo de médias específicas de domínio, tornando a estimativa mais precisa.

6 A abreviatura *pm* significa pessoa-mês de trabalho.



ricos ou (quando tudo o mais falhar) intuição, estimam-se valores de tamanho otimista, mais provável e pessimista para cada função ou contagem para cada valor do domínio de informações. Quando um intervalo de valores é especificado, é fornecida uma indicação implícita do grau de incerteza.

Um valor esperado ou de três pontos pode então ser calculado. O *valor esperado* para a variável de estimativa (tamanho) S pode ser calculado como uma média ponderada das estimativas otimista (S_{opt}), mais provável (S_m) e pessimista (S_{pess}). Por exemplo,

$$S = \frac{S_{opt} + 4S_m + S_{pess}}{6} \quad (26.1)$$

oferece a maior credibilidade da estimativa “mais provável” e segue uma distribuição beta de probabilidade. Assumimos que há uma probabilidade muito pequena de que o tamanho real fique fora dos valores otimista e pessimista.

Uma vez determinado o valor esperado para a variável de estimativa, aplicam-se os dados de produtividade histórica de LOC ou FP. As estimativas estão corretas? A única resposta razoável a essa pergunta é: “Não se pode ter certeza”. Qualquer técnica de estimativa, não importa quão sofisticada seja, deve passar por uma verificação cruzada com outra abordagem. Mesmo assim, o bom senso e a experiência devem prevalecer.

26.6.3 Um exemplo de estimativa baseada em LOC

Como exemplo das técnicas de estimativa em LOC e FP baseadas em problema, vamos considerar um pacote de software a ser desenvolvido para uma aplicação de projeto auxiliado por computador para componentes mecânicos. O software deve ser executado em uma estação de trabalho de engenharia e deve ter interface com vários periféricos gráficos incluindo um mouse, um teclado, um monitor colorido de alta resolução e uma impressora laser. Pode ser formulada uma definição preliminar do escopo do software:



Muitas aplicações modernas residem em uma rede ou fazem parte de uma arquitetura cliente-servidor. Portanto, certifique-se de que suas estimativas incluem a mão de obra requerida para desenvolver software de “infraestrutura”.

O software CAD mecânico aceitará dados geométricos bidimensionais e tridimensionais fornecidos por um engenheiro. O engenheiro vai interagir e controlar o sistema CAD através de uma interface de usuário que irá exibir características de um bom design de interface homem/máquina. Todos os dados geométricos e outras informações de suporte serão mantidos em uma base de dados CAD. Serão desenvolvidos módulos de análise de projeto para produzir a saída requerida, a ser exibida em uma variedade de dispositivos gráficos. O software será projetado para controlar e interagir com dispositivos periféricos que incluem um mouse, um teclado, impressora laser e um plotter.

Essa definição de escopo é preliminar – não é delimitada. Cada sentença deverá ser expandida para proporcionar detalhe concreto e limites quantitativos. Por exemplo, antes de iniciar a estimativa, o planejador deve determinar o que significa “características de projeto de uma boa interface homem/máquina” ou qual deverão ser o tamanho e a sofisticação da “base de dados CAD”.

Para nossos propósitos, suponha que ocorreu um refinamento adicional e que as principais funções do software listadas na Figura 26.2 estão identificadas. Seguindo a técnica de decomposição para LOC, é desenvolvida uma tabela de estimativas (Figura 26.2). Para cada função é desenvolvido um intervalo de estimativas LOC. Por exemplo, o intervalo de estimativas LOC para a função de análise geométrica 3-D é otimista, 4.600 LOC; mais provável, 6.900 LOC; e pessimista, 8.600 LOC. Aplicando a Equação 26.1, o valor esperado para a função de análise geométrica 3D é 6.800 LOC. Outras estimativas são obtidas de forma semelhante. Somando verticalmente na coluna de estimativa LOC, obtém-se uma estimativa de 33.200 linhas de código para o sistema CAD.

Um exame de dados históricos indica que a produtividade média organizacional para sistemas desse tipo é de 620 LOC/pm. Com base em um valor bruto da mão de obra de \$ 8 mil por mês, o custo por linha de código é de aproximadamente \$ 13. Com base na estimativa LOC e dados históricos de produtividade, o custo total estimado do projeto é de \$ 431 mil e o esforço estimado é de 54 pessoas-mês.⁷



Não ceda à tentação de usar esse resultado como sua estimativa de projeto. Você deverá derivar um outro resultado usando uma abordagem diferente.

⁷ As estimativas são arredondadas para o próximo \$ 1.000 e pessoa-mês. Precisão maior não é necessária nem realística, dadas as limitações da precisão da estimativa.

FIGURA 26.2

Tabela de estimativas para o método LOC

Função	LOC estimado
Interface de usuário e recurso de controle	2.300
Análise geométrica bidimensional	5.300
Análise geométrica tridimensional	6.800
Gerenciamento de base de dados	3.350
Recursos de visualização da computação gráfica	4.950
Função de controle de periféricos	2.100
Módulos de análise do projeto	8.400
<i>Linhas de código estimadas</i>	<i>33.200</i>

CASASEGURA



Estimativa

Cena: Escritório de Doug Miller no início do planejamento.

Atores: Doug Miller (gerente da equipe de engenharia de software do *CasaSegura*), Vinod Raman, Jamie Lazar e outros membros da equipe de engenharia de software.

Conversa:

Doug: Precisamos desenvolver uma estimativa de esforço para projeto e depois temos de definir um microcronograma para o primeiro incremento e um macrocronograma para os demais incrementos.

Vinod (acenando afirmativamente): Ok, mas não definimos nenhum incremento ainda.

Doug: Sim, mas é por isso que precisamos fazer estimativas.

Jamie (contrariado): Você quer saber quanto tempo vamos gastar para fazer isso?

Doug: Veja o que eu preciso. Primeiro, precisamos decompor funcionalmente o software *CasaSegura* em alto nível... Depois temos de estimar o número de linhas de código que cada função terá... Depois...

Jamie: Espera um pouco! Como você acha que vamos fazer isso?

Vinod: Já fizemos em projetos anteriores. Você começa com casos de uso, determina a funcionalidade requerida para implementar cada um, arrisca um palpite quanto ao número de LOC para cada parte da função. A melhor abordagem é que todos façam isso de forma independente e depois comparemos os resultados.

Doug: Ou você pode fazer uma decomposição funcional do projeto inteiro.

Jamie: Mas isso vai demorar uma eternidade e precisamos começar.

Vinod: Não... Isso pode ser feito em algumas horas... Na verdade, nesta manhã.

Doug: Concorde... Não podemos esperar exatidão, apenas uma vaga ideia do tamanho que o *CasaSegura* terá.

Jamie: Acho que deveríamos apenas estimar o esforço... Só isso.

Doug: Faremos isso também. Depois usaremos ambas as estimativas como verificação cruzada.

Vinod: Então vamos fazer...

26.6.4 Um exemplo de estimativa baseada em FP

A decomposição para estimativa baseada em FP concentra-se em valores do domínio da informação em lugar das funções de software. De acordo com a tabela apresentada na Figura 26.3, você estimaria entradas, saídas, consultas, arquivos e interfaces externas para o software CAD. É computado um valor de FP usando a técnica discutida no Capítulo 23. Para os propósitos dessa estimativa, o fator de peso da complexidade é adotado como médio. A Figura 26.3 apresenta os resultados dessa estimativa.

FIGURA 26.3

Estimando valores do domínio de informações

Valor do domínio	Esti- Saídas	Estimativa Consulta	Estimativa computada	FB Peso computado	
Número de entradas externas	20	24	30	24	4
Número de saídas externas	12	15	22	16	5
Número de consultas externas	16	22	28	22	5
Número de arquivos lógico internos	4	4	5	4	10
Número de arquivos de interface externos	2	2	3	2	7
<i>Contagem total</i>					320

É estimado cada um dos fatores de peso da complexidade; o fator de ajuste do valor é calculado conforme descrito no Capítulo 23:

Fator	Valor
Backup e recuperação	4
Comunicações de dados	2
Processamento distribuído	0
Desempenho crítico	4
Ambiente operacional existente	3
Entrada de dados on-line	4
Transações de entrada em múltiplas telas	5
Arquivos mestres atualizados on-line	3
Complexidade dos valores dos domínios de informação	5
Complexidade do processamento interno	5
Código projetado para reutilização	4
Conversão/instalação no projeto	3
Instalações múltiplas	5
Aplicação projetada para alteração	5
Fator de ajuste de valor	1,17

Por fim, é obtido o número estimado de FP:

$$FP_{\text{estimado}} = \text{contagem total} \times [0,65 + 0,01 \times \Sigma(F_i)] = 375$$

A produtividade média organizacional para sistemas desse tipo é de 6,5 FP/pm. Com base em um valor bruto de mão de obra de \$ 8 mil por mês, o custo por FP é aproximadamente de \$ 1.230. Com base na estimativa FP e nos dados de produtividade histórica, o custo total estimado do projeto é de \$ 461 mil, e a estimativa de esforço é de 58 pessoas-mês.

26.6.5 Estimativas baseadas em processo

A técnica mais comum para estimativa de projeto é baseá-la no processo a ser usado. Isto é, o processo é decomposto em um conjunto relativamente pequeno de tarefas, e é estimado o trabalho necessário para executar cada tarefa.

FIGURA 26.4

Tabela de estimativa baseada em processo

Atividade	CC	Planejamento	Análise de risco	Engenharia		Liberação da versão		AC	Totais
Tarefa				Análises	Design	Code	Test		
Função									
UICF				0,50	2,50	0,40	5,00	n/a	8,40
2DGA				0,75	4,00	0,60	2,00	n/a	7,35
3DGA				0,50	4,00	1,00	3,00	n/a	8,50
CGDF				0,50	3,00	1,00	1,50	n/a	6,00
DBM				0,50	3,00	0,75	1,50	n/a	5,75
PCF				0,25	2,00	0,50	1,50	n/a	4,25
DAM				0,50	2,00	0,50	2,00	n/a	5,00
Totais	0,25	0,25	0,25	3,50	20,50	4,50	16,50		46,00
% de mão de obra	1%	1%	1%	8%	45%	10%	36%		

CC = comunicação com o cliente AC = avaliação pelo cliente

Assim como as técnicas baseadas em problemas, a estimativa baseada em processo começa com um delineamento das funções de software obtidas do escopo do projeto. Para cada função, deve ser executada uma série de atividades estruturais. As funções e atividades estruturais relacionadas⁸ podem ser representadas como parte de uma tabela similar à da Figura 26.4.



Se houver tempo suficiente, use maior detalhamento ao especificar tarefas na Figura 26.4. Por exemplo, subdivida a análise em suas tarefas principais e estime cada uma separadamente.

Uma vez combinadas as funções do problema e atividades de processo, você pode estimar o esforço (por exemplo, pessoas-mês) necessário para executar cada atividade do processo de software, para cada função de software. Esses dados constituem a matriz central da tabela da Figura 26.4. São então aplicados os valores médios de preço de mão de obra (isto é, custo/unidade de esforço) ao esforço estimado para cada atividade de processo. É muito provável que os valores da mão de obra irão variar para cada tarefa. O pessoal mais experiente se envolve mais intensamente nas primeiras atividades estruturais e geralmente são mais caros do que o pessoal menos experiente envolvido na construção e liberação das versões.

Custos e esforço para cada função e para cada atividade estrutural são calculados, como última etapa. Se a estimativa baseada em processo é feita independentemente da estimativa baseada em LOC ou FP, temos agora duas ou três estimativas para custo e esforço que podem ser comparadas e reconciliadas. Se ambos os conjuntos de estimativas apresentarem concordância razoável, há boas razões para acreditar que as estimativas são confiáveis. Por outro lado, se os resultados dessas técnicas de decomposição mostrarem pouca concordância, deve ser executada uma investigação e análise mais profunda.

26.6.6 Um exemplo de estimativa baseada em processo

Para ilustrar o uso de estimativas baseadas em processo, considere o software CAD introduzido na Seção 26.6.3. A configuração do sistema e todas as funções de software permanecem inalteradas e são indicadas pelo escopo do projeto.

De acordo com a tabela baseada em processo mostrada na Figura 26.4, são feitas as estimativas de esforço (em pessoas-mês) para cada atividade de engenharia de software para cada função do software CAD (abreviado para maior simplicidade). As atividades de engenharia e construção subdividem-se nas principais tarefas de engenharia de software da tabela. São feitas estimativas aproximadas de esforço para comunicação com o cliente, planejamento e análise de riscos. Os totais são colocados na parte inferior da tabela. Os totais horizontais e verticais fornecem uma indicação do esforço estimado necessário para análise, projeto, codificação e

“É melhor entender os fundamentos de uma estimativa antes de utilizá-la.”

**Barry Boehm e
Richard Fairley**

⁸ As atividades estruturais escolhidas para este projeto diferem um pouco das atividades genéricas discutidas no Capítulo 2. São elas: comunicação com o cliente (*customer communication* – CC), planejamento, análise de risco, engenharia e construção/liberação das versões.

teste. Devemos observar que 53% de todo o esforço são gastos nas tarefas de engenharia inicial (análise de requisitos e projeto), indicando a importância relativa deste trabalho.

Com base na taxa média bruta de mão de obra de \$ 8 mil por mês, o custo total estimado do projeto é de \$ 368 mil e o esforço estimado é de 46 pessoas-mês. Se você quiser, as taxas de mão de obra podem ser associadas a cada atividade estrutural ou a cada tarefa de engenharia de software e calculadas separadamente.

26.6.7 Estimativa com casos de uso

Conforme afirmamos na Parte 2 deste livro, os casos de uso proporcionam a uma equipe de software informações sobre o escopo do software e os requisitos. No entanto, é problemático desenvolver uma abordagem de estimativa com casos de uso pelas seguintes razões [Smi99]:

- Casos de uso são descritos por meio de muitos formatos e estilos diferentes – não há um formato-padrão.
- Casos de uso representam uma visão externa (a visão do usuário) do software e podem, portanto, ser escritos em muitos diferentes níveis de abstração.
- Casos de uso não tratam da complexidade das funções e características que são descritas.
- Casos de uso podem descrever comportamento complexo (por exemplo, interações) que envolvem muitas funções e características.

Diferentemente de uma LOC ou ponto de função, um “caso de uso” pode precisar de meses de trabalho enquanto o caso de uso de outra pessoa pode ser implementado em um dia ou dois.

Embora muitos pesquisadores tenham considerado os casos de uso uma informação de estimativa, até agora não existe nenhum método de estimativa comprovado.⁹ Smith [Smi99] sugere que os casos de uso podem ser empregados para estimativa, mas somente se forem considerados no contexto da “hierarquia estrutural” em que são usados para descrever.

Smith afirma que qualquer nível dessa hierarquia estrutural pode ser descrito por não mais do que 10 casos de uso. Cada um poderia abranger não mais do que trinta cenários distintos. Obviamente, casos de uso que descrevem um grande sistema são escritos com um nível muito mais alto de abstração (e representam consideravelmente bem mais esforço de desenvolvimento) do que aqueles escritos para descrever um único subsistema. Portanto, antes que os casos de uso possam ser empregados para estimativas, é estabelecido o nível da hierarquia estrutural, determinado o tamanho médio (em páginas) de cada caso, definido o tipo de software (por exemplo, em tempo real, corporativo, engenharia/científico, WebApp, embarcado), e é considerada uma arquitetura primitiva para o sistema. Uma vez estabelecidas essas características, podem ser utilizados dados empíricos para estabelecer o número estimado de LOC ou FP por caso de uso (para cada nível de hierarquia). Usam-se então dados históricos para calcular o esforço necessário para desenvolver o sistema.

Para ilustrar como essa computação pode ser feita, considere a seguinte relação:¹⁰

$$\text{LOC estimado} = N \times \text{LOC}_{\text{média}} + [(S_d/S_h - 1) + (P_d/P_h - 1)] \times \text{LOC}_{\text{ajustado}} \quad (26.2)$$

em que

- N = número atual de casos de uso
- $\text{LOC}_{\text{média}}$ = média histórica de LOC por caso de uso para esse tipo de subsistema
- $\text{LOC}_{\text{ajustado}}$ = representa um ajuste baseado em n por cento de $\text{LOC}_{\text{média}}$ em que n é definido localmente e representa a diferença entre esse projeto e a “média” dos outros projetos

⁹ Trabalho recente sobre a derivação de *pontos de caso de uso* [Cle06] pode finalmente levar a uma abordagem prática de estimativa por meio de casos de uso.

¹⁰ É importante observar que a Expressão (26.2) é usada apenas para fins de ilustração. Assim como todos os modelos de estimativa, ela pode ser validada localmente para que possa ser usada com segurança.

? Por que é difícil desenvolver uma técnica de estimativa usando casos de uso?

- S_a = cenários atuais por caso de uso
 S_h = cenários médios por caso de uso para esse tipo de subsistema
 P_a = número atual de páginas por caso de uso
 P_h = número médio de páginas por caso de uso para esse tipo de subsistema

A Expressão (26.2) poderia ser usada para desenvolver uma estimativa aproximada do número de LOC com base no número real de casos de uso ajustado pelo número de cenários e tamanhos de página dos casos. O ajuste representa até n por cento da média histórica de LOC por caso de uso.

26.6.8 Exemplo de estimativa baseada em caso de uso

O software CAD apresentado na Seção 26.6.3 é composto de três grupos de subsistemas: subsistema da interface do usuário (inclui UICF), grupo do subsistema de engenharia (inclui os subsistemas 2DGA, 3DGA e DAM) e grupo do subsistema de infraestrutura (inclui os subsistemas CGDF e PCF). Seis casos de uso descrevem o subsistema de interface de usuário. Cada caso é descrito por não mais do que 10 cenários e tem um tamanho médio de 6 páginas. O grupo do subsistema de engenharia é descrito por 10 casos de uso (considerados pertencentes a um nível mais alto na hierarquia estrutural). Cada um deles tem não mais de 20 cenários associados e um tamanho médio de 8 páginas. Por fim, o grupo do subsistema infraestrutura é descrito por 5 casos de uso com uma média de apenas seis cenários e um tamanho médio de 5 páginas.

Por meio das relações descritas na Expressão (26.2) com $n = 30\%$, calcula-se a tabela da Figura 26.5. Considerando a primeira linha da tabela, dados históricos indicam que o software da interface de usuário (UI) requer uma média de 800 LOC por caso de uso quando este não tiver mais do que 12 cenários e for descrito em menos de 5 páginas. Esses dados se adaptam razoavelmente bem para o sistema CAD. Portanto, a estimativa LOC para o subsistema de interface de usuário é calculado usando-se a Expressão (26.2). Pela mesma abordagem, são feitas estimativas para os grupos de subsistemas de engenharia e infraestrutura. A Figura 26.5 resume as estimativas e indica que o tamanho total do CAD é estimado em 42.500 linhas de código (LOC).

Usando 620 LOC/pm como produtividade média para sistemas desse tipo e um custo bruto de mão de obra de \$ 8 mil por mês, o custo por linha de código é de aproximadamente \$ 13. Com base na estimativa de casos de uso e dados de produtividade histórica, o custo total estimado do projeto é de \$ 552 mil e o esforço estimado é de 68 pessoas-mês.

FIGURA 26.5

Estimativa de caso de uso

	casos de uso	cenários	páginas	cenários	páginas	LOC	LOC estimado
Subsistema de interface de usuário	6	10	6	12	5	560	3.366
Grupo do subsistema de engenharia	10	20	8	16	8	3100	31.233
Grupo do subsistema de infraestrutura	5	6	5	10	6	1650	7.970
Total estimado de LOC							42.568

"Métodos complicados podem não produzir uma estimativa exata, particularmente quando os desenvolvedores podem incorporar sua própria intuição na estimativa."

Philip Johnson e outros

26.6.9 Reconciliando estimativas

As técnicas de estimativa discutidas nas seções anteriores resultam em múltiplas estimativas que devem ser reconciliadas para produzir uma única estimativa de esforço, duração de projeto ou custo. Para tanto, vamos considerar novamente o software CAD introduzido na Seção 26.6.3.

O esforço total estimado para o software CAD varia desde um valor baixo de 46 pessoas-mês (derivado de uma abordagem de estimativa baseada em processo) até um valor alto de 68 pessoas-mês (derivado de uma estimativa de caso de uso). A estimativa média (com as quatro abordagens) é de 56 pessoas-mês. A variação em relação à estimativa média é de aproximadamente 18% para baixo e 21% para cima.

O que acontece quando a concordância entre as estimativas é baixa? A resposta a essa questão requer uma reavaliação das informações usadas para fazer as estimativas. Estimativas que divergem muito podem ser atribuídas a uma de duas causas: (1) o escopo do projeto não é entendido adequadamente ou foi mal interpretado pelo planejador ou (2) os dados de produtividade usados para as técnicas de estimativa baseada em problema são inadequados para a aplicação, obsoletos (porque não refletem mais com precisão a organização de engenharia de software) ou foram mal aplicados. Você deverá determinar a causa da divergência e reconciliar as estimativas.



Técnicas automáticas de estimativa para projetos de software

As ferramentas de estimativa automáticas permitem ao planejador estimar o custo e o esforço e executar análises do tipo “e se...” para variáveis importantes do projeto como data de entrega ou equipe. Embora existam muitas ferramentas de estimativas automáticas, todas apresentam as mesmas características gerais e executam as seis funções genéricas a seguir [Jon96]:

1. *Dimensionamento dos entregáveis do projeto.* O “tamanho” de um ou mais artefatos de software é estimado. Artefatos incluem a representação externa do software (por exemplo, tela, relatórios), o próprio software (por exemplo, KLOC), funcionalidade fornecida (por exemplo, pontos de função) e informações descritivas (por exemplo, documentos).
2. *Seleção de atividades de projeto.* É selecionada a estrutura apropriada de processo e especificado o conjunto de tarefas de engenharia de software.
3. *Previsão do número de pessoas.* É especificado o número de pessoas que estarão disponíveis para o trabalho. Esse é um dado importante porque a relação entre pessoas disponíveis e trabalho (esforço previsto) é altamente não linear.

4. *Previsão do esforço de software.* As ferramentas de estimativa empregam um ou mais modelos (Seção 26.7) que relacionam o tamanho dos entregáveis do projeto com o esforço necessário para produzi-los.

5. *Previsão do custo do software.* Dados os resultados da etapa 4, os custos podem ser calculados alocando-se valores da mão de obra às atividades de projeto citadas na etapa 2.

6. *Previsão dos cronogramas do software.* Ao conhecermos esforço, número de pessoas e atividades de projeto, podemos produzir um esboço de cronograma alocando mão de obra às atividades de engenharia de software com base nos modelos recomendados para distribuição do esforço discutidos mais adiante neste capítulo.

Quando são aplicadas diferentes ferramentas de estimativa para os mesmos dados de projeto, pode ser encontrada uma variação razoavelmente grande nos resultados estimados. Mais importante ainda, os valores previstos muitas vezes são significativamente diferentes dos valores reais. Isso reforça a noção de que o produto das ferramentas de estimativas deverá ser usado como “ponto de dados” por meio do qual são obtidas as estimativas – não como a única fonte para uma estimativa.

INFORMAÇÕES

26.7 MODELOS EMPÍRICOS DE ESTIMATIVA

Um modelo de estimativa para software usa fórmulas derivadas empiricamente para prever o esforço como uma função de LOC ou FP.¹¹ Valores para LOC ou FP são estimados usando a abordagem descrita nas Seções 26.6.3 e 26.6.4. Em vez de utilizarmos as tabelas descritas naquelas seções, os valores resultantes para LOC ou FP são anexados ao modelo de estimativa.

Os dados empíricos que suportam muitos modelos de estimativa são obtidos de uma amostragem limitada de projetos. Por essa razão, nenhum modelo de estimativa é apropriado para todas as classes de software e todos os ambientes de desenvolvimento. Portanto, você deverá usar os resultados obtidos desses modelos com muito critério.

Um modelo de estimativa deverá ser calibrado para refletir condições locais. O modelo deverá ser testado aplicando-se dados coletados de projetos completos, anexando os dados no modelo e comparando resultados reais com os previstos. Se a concordância for baixa, o modelo deve ser ajustado e retestado antes de ser usado.

PONTO-CHAVE

Um modelo de estimativa reflete a população de projetos com base na qual foi obtido. Portanto, o modelo é sensível ao domínio.

¹¹ Um modelo empírico que utiliza casos de uso como variável independente é sugerido na Seção 26.6.6. No entanto, apenas alguns apareceram na literatura até agora.

26.7.1 Estrutura dos modelos de estimativa

Um modelo típico de estimativa é obtido por meio da análise de regressão sobre dados coletados de projetos de software anteriores. A estrutura geral desses modelos toma a forma [Mat94]

$$E = A + B \times (e_v)^c \quad (26.3)$$

em que A , B e C são constantes obtidas empiricamente, E é o esforço em pessoas-mês e e_v é a variável de estimativa (LOC ou FP). Além da relação descrita na Equação (26.3), a maioria dos modelos de estimativa tem alguma forma de componente de ajuste de projeto que permite que E seja ajustado por outras características do projeto (por exemplo, complexidade do problema, experiência da equipe, ambiente de desenvolvimento). Entre os muitos modelos de estimativas orientados para LOC propostos na literatura estão



Nenhum desses modelos deverá ser usado sem uma cuidadosa calibração com o seu ambiente.

$$E = 5,2 \times (\text{KLOC})^{0,91}$$

Modelo Walston-Felix

$$E = 5,5 + 0,73 \times (\text{KLOC})^{1,16}$$

Modelo Bailey-Basili

$$E = 3,2 \times (\text{KLOC})^{1,05}$$

Modelo Boehm simples

$$E = 5,288 \times (\text{KLOC})^{1,047}$$

Modelo Doty para KLOC > 9

Também foram propostos modelos orientados a FP. São eles

$$E = -91,4 + 0,355 \text{ FP}$$

Modelo Albrecht e Gaffney

$$E = -37 + 0,96 \text{ FP}$$

Modelo Kemerer

$$E = -12,88 + 0,405 \text{ FP}$$

Modelo de regressão de pequeno projeto

Um rápido exame desses modelos indica que cada um produzirá um resultado diferente para os mesmos valores de LOC ou FP. A implicação é clara. Modelos de estimativa devem ser calibrados para as necessidades locais!

26.7.2 O modelo COCOMO II

WebRef

Informações detalhadas sobre COCOMO II, incluindo software para download, podem ser obtidas no endereço sunset.usc.edu/research/COCOMOII/cocomo_main.html.

Em seu livro clássico sobre “economia da engenharia de software” (*software engineering economics*), Barry Boehm [Boe81] introduziu uma hierarquia de modelos de estimativa de software com o nome COCOMO, que significa *CONstructive COSt MOdel* (modelo construtivo de custo). O COCOMO original tornou-se um dos modelos de estimativa de custo de software mais amplamente usado e discutido na indústria. Ele evoluiu para um modelo de estimativa mais abrangente, chamado COCOMOII [Boe00]. Assim como seu predecessor, o COCOMO II é na realidade uma hierarquia de modelos de estimativa que trata das seguintes áreas:

- *Modelo de composição de aplicação.* Usado durante os primeiros estágios da engenharia de software, em que protótipo das interfaces de usuário, a consideração da interação de software e sistema, a avaliação do desempenho e a avaliação da maturidade da tecnologia são de suma importância.
- *Modelo de estágio do início do projeto.* Usado quando os requisitos tiverem sido estabilizados e a arquitetura básica de software tiver sido estabelecida.
- *Modelo de estágio pós-arquitetura.* Usado durante a construção do software.

Assim como todos os modelos de estimativa para software, os modelos COCOMO II requerem informações de tamanho. Há disponíveis três diferentes opções como parte da hierarquia de modelo: pontos de objeto, pontos de função e linhas de código-fonte.

O modelo de composição de aplicação COCOMO II utiliza pontos de objeto e é ilustrado nos próximos parágrafos. Devemos observar que há também disponíveis outros modelos de estimativa mais sofisticados (usando FP e KLOC) como parte do COCOMO II.

Assim como os pontos de função, o *ponto de objeto* é uma medida indireta de software calculada por meio de contagens dos números de (1) telas (na interface do usuário), (2) relatórios e



FIGURA 26.6

Peso da complexidade para tipos de objeto

Fonte: (Boe96)

Tipo de objeto	Peso da complexidade		
	Simple	Médio	Difícil
Tela	1	2	3
Relatório	2	5	8
Componente 3GL			10

(3) componentes que podem ser necessários para construir a aplicação. Cada instância de objeto (por exemplo, uma tela ou um relatório) é classificada em um dentre três níveis de complexidade (simple, médio ou difícil) usando critérios sugeridos por Boehm [Boe96]. Essencialmente, a complexidade é uma função da quantidade e origem das tabelas de dados-cliente e servidor necessárias para gerar a tela ou relatório e o número de visualizações ou seções apresentadas como parte da tela ou relatório.

Uma vez determinada a complexidade, os números de telas, relatórios e componentes são ponderados de acordo com a tabela da Figura 26.6. A contagem de pontos de objeto é então determinada multiplicando-se o número original de instâncias de objeto pelo fator de peso na figura e somando para obter o total da contagem de pontos de objeto. Quando deve ser aplicado desenvolvimento baseado em componente ou reutilização de software em geral, é estimada a porcentagem de reutilização (% reuso) e é ajustada a contagem de pontos de objeto:

$$NOP = (\text{pontos de objeto}) \times [(100 - \% \text{reuso})/100]$$

em que NOP é definido como novos pontos de objeto.

Para derivar uma estimativa de esforço com base no valor calculado para NOP, deve ser derivada uma “taxa de produtividade”. A Figura 26.7 apresenta a taxa de produtividade

$$PROD = \frac{NOP}{\text{pessoa-mês}}$$

para diferentes níveis de experiência do desenvolvedor e maturidade do ambiente de desenvolvimento.

Uma vez determinada a taxa de produtividade, calcula-se a estimativa de esforço do projeto usando

$$\text{Esforço estimado} = \frac{NOP}{PROD}$$

Em modelos COCOMO II mais avançados,¹² é necessária uma variedade de fatores de escala, custos e procedimentos de ajuste. Uma discussão completa desses tópicos está além do escopo deste livro. Se você estiver interessado, veja [Boe00] ou visite o site do COCOMO II.

FIGURA 26.7

Taxa de produtividade para pontos de objeto

Fonte: (Boe96)

Experiência/capacidade do desenvolvedor	Muito baixa	Baixa	Nominal	Alta	Muito alta
Maturidade/capacidade do ambiente	Muito baixa	Baixa	Nominal	Alta	Muito alta
PROD	4	7	13	25	50

¹² Conforme afirmamos anteriormente, esses modelos usam contagens FP e KLOC para a variável tamanho.

26.7.3 A Equação do software

A *equação do software* [Put92] é um modelo dinâmico multivariável que assume uma distribuição específica de esforço durante a vida de um projeto de desenvolvimento de software. O modelo foi derivado dos dados de produtividade coletados em mais de 4 mil projetos de software. Com base nesses dados, derivamos um modelo de estimativa da forma

$$E = \frac{\text{LOC} \times B^{0,333}}{P^3} \times \frac{1}{t^4} \quad (26.4)$$

em que

E = esforço em pessoas-mês ou pessoas-ano

t = duração do projeto em meses ou anos

B = “fator de habilidades especiais”¹³

P = “parâmetro de produtividade” que reflete: maturidade geral do processo e práticas gerenciais, a amplitude na qual são usadas as boas práticas de engenharia de software, o nível de linguagens de programação usado, o estado do ambiente de software, as habilidades e experiência da equipe de software e a complexidade da aplicação

Os valores típicos podem ser $P = 2$ mil para desenvolvimento de software embutido em tempo real, $P = 10$ mil para software de telecomunicações e sistemas, e $P = 28$ mil para aplicações de sistemas comerciais. O parâmetro de produtividade pode ser derivado para condições locais por meio de dados históricos coletados de trabalhos de desenvolvimento executados no passado.

Você deve observar que a equação de software tem dois parâmetros independentes: (1) uma estimativa do tamanho (em LOC) e (2) uma indicação da duração do projeto em meses corridos ou anos.

Para simplificarem o processo de estimativa e usarem uma forma mais comum para o modelo de estimativa, Putnam e Myers [Put92] sugerem uma série de equações derivadas da equação do software. O tempo de desenvolvimento mínimo é definido como

$$t_{\min} = 8,14 \frac{\text{LOC}}{P^{0,43}} \text{ em meses para } t_{\min} > 6 \text{ meses} \quad (26.5a)$$

$$E = 180 B t^3 \text{ em pessoas-mês para } E \geq 20 \text{ pessoas-mês} \quad (26.5b)$$

Note que t na Equação (26.5b) está representado em anos.

Usando a Equação (26.5) com $P = 12.000$ (o valor recomendado para software científico) para o software CAD discutido anteriormente neste capítulo,

$$t_{\min} = 8,14 \times \frac{33.200}{12.000^{0,43}} = 12,6 \text{ meses corridos}$$

$$E = 180 \times 0,28 \times (1,05)^3 = 58 \text{ pessoas-mês}$$

Os resultados da equação do software correspondem favoravelmente com a estimativa desenvolvida na Seção 26.6. Assim como o modelo COCOMO discutido na Seção 26.7.2, a equação do software continua evoluindo. Uma discussão mais detalhada dessa abordagem de estimativa pode ser encontrada em [Put97b].

WebRef

Informações sobre ferramentas de estimativas de custo de software originadas da equação do software podem ser encontradas em www.qsm.com.

¹³ B aumenta lentamente à medida que cresce a “necessidade de integração, teste, garantia de qualidade, documentação e habilidades de gerenciamento” [Put92]. Para pequenos programas (KLOC = 5 a 15), $B = 0,16$. Para programas maiores do que 70 KLOC, $B = 0,39$.

26.8 ESTIMATIVA PARA PROJETOS ORIENTADOS A OBJETO

É conveniente suplementar os métodos convencionais de estimativa de custo de software com uma técnica criada explicitamente para software orientado a objeto. Lorenz e Kidd [Lor94] sugerem a seguinte abordagem:

1. Desenvolva estimativas usando decomposição de esforço, análise FP e qualquer outro método para aplicações convencionais.
2. Usando o modelo de requisitos (Capítulo 6), desenvolva casos de uso e determine uma contagem. Reconheça que o número de casos de uso pode mudar à medida que o projeto avança.
3. Com base no modelo de requisitos, determine o número de classes-chave (chamadas de classes de análise no Capítulo 6).
4. Classifique o tipo de interface para a aplicação e desenvolva um multiplicador para classes de apoio:

Tipo de Interface	Multiplicador
Sem GUI	2,0
Interface de usuário baseada em texto	2,25
GUI	2,5
GUI Complexa	3,0

Multiplique o número de classes-chave (passo 3) pelo multiplicador para obter uma estimativa do número de classes de apoio.

5. Multiplique o número total de classes (classes-chave + classes de apoio) pelo número médio de unidades de trabalho por classe. Lorenz e Kidd sugerem 15 a 20 pessoas-dia por classe.
6. Faça uma verificação cruzada da estimativa baseada em classe multiplicando o número médio de unidades de trabalho por caso de uso.

26.9 TÉCNICAS ESPECIALIZADAS DE ESTIMATIVA

As técnicas de estimativa discutidas nas Seções 26.6 a 26.8 podem ser empregadas em qualquer projeto de software. No entanto, quando uma equipe de software encontra um prazo de projeto extremamente curto (semanas em vez de meses), no qual é provável que haja uma sequência contínua de alterações, o planejamento do projeto em geral e a estimativa em particular deverão ser abreviados.¹⁴ Nas próximas seções, examinaremos duas técnicas especializadas de estimativa.

26.9.1 Estimativa para desenvolvimento ágil

Como os requisitos para um projeto ágil (Capítulo 3) são definidos por uma série de cenários de usuário (por exemplo, “histórias” em Programação Extrema), é possível desenvolver uma abordagem de estimativa informal, razoavelmente disciplinada e significativa no contexto de planejamento de projeto para cada incremento de software. A estimativa para projetos ágeis usa uma abordagem de decomposição que abrange os seguintes passos:

1. Cada cenário de usuário (o equivalente a um minicaso de uso criado bem no início de um projeto por usuários finais ou outros interessados) é considerado separadamente para fins de estimativa.

 Como as estimativas são desenvolvidas quando aplicado um processo ágil?

¹⁴ “Abreviado” não significa eliminado. Mesmo os projetos de curta duração deverão ser planejados, e a estimativa é o fundamento do planejamento.

PONTO-CHAVE

No contexto de estimativa para projetos ágeis, “volume” é uma estimativa do tamanho total de um cenário de usuário em LOC ou FP.

2. O cenário é decomposto em uma série de tarefas de engenharia de software que serão necessárias para desenvolvê-lo.
- 3a. O esforço necessário para cada tarefa é estimado separadamente. Nota: A estimativa pode ser baseada em dados históricos, em um modelo empírico ou na “experiência”.
- 3b. Como alternativa, o “volume” do cenário pode ser estimado em LOC, FP ou alguma outra medida orientada para volume (por exemplo, contagem de caso de uso).
- 4a. As estimativas para cada tarefa são somadas para criar uma estimativa para o cenário.
- 4b. Como alternativa, o volume estimado para o cenário é traduzido em esforço usando dados históricos.
5. As estimativas de esforço para todos os cenários implementados para determinado incremento de software são somadas para desenvolver a estimativa de esforço para o incremento.

Como a duração necessária para o desenvolvimento de um incremento de software é bastante curta (tipicamente três a seis semanas), essa abordagem de estimativa serve a duas finalidades: (1) para a certificação de que o número de cenários a ser incluídos no incremento corresponde aos recursos disponíveis, e (2) para estabelecer uma base para alocar esforço na medida em que o incremento é desenvolvido.

26.9.2 Estimativa para projetos para WebApp

Os projetos para WebApp muitas vezes adotam o modelo de processo ágil. Uma medida modificada de pontos de função, acoplada com as etapas resumidas na Seção 26.9.1, pode ser usada para desenvolver uma estimativa para a WebApp. Roetzheim [Roe00] sugere a seguinte abordagem ao adaptar pontos de função para estimativa de WebApp:

- *Entradas (Inputs)*. Cada tela de entrada ou formulário [form] (por exemplo, CGI ou Java), cada tela de manutenção e, se você usa uma metáfora de conjunto de guias em algum lugar [tab notebook], cada guia [tab].
- *Saídas (Outputs)*. Cada página da Web estática, cada script de página da Web dinâmica (por exemplo, ASP, ISAPI ou outro script DHTML) e cada relatório (baseado na Web ou de natureza administrativa).
- *Tabelas (Tables)*. Cada tabela lógica na base de dados e, se você estiver usando XML para armazenar dados em um arquivo, cada objeto XML (ou coleção de atributos XML).
- *Interfaces (Interfaces)*. Retêm sua definição como arquivos lógicos (por exemplo, formatos únicos de registro) em nossas fronteiras de saída do sistema.
- *Consultas (Queries)*. Publicadas externamente ou usam uma interface orientada por mensagem. Um exemplo típico são as referências externas DCOM ou COM.

Pontos de função (interpretados da maneira descrita) é um indicador razoável do volume para uma WebApp.

Mendes e seus colegas [Men01] sugerem que o volume de uma WebApp é mais bem determinado coletando-se medidas (chamadas de “variáveis de previsão”) associadas à aplicação (por exemplo, número de páginas, contagem de mídias, número de funções), suas características de páginas da Web (por exemplo, complexidade da página, complexidade de vínculo, complexidade gráfica), características da mídia (por exemplo, duração da mídia) e características funcionais (por exemplo, tamanho do código, tamanho do código reutilizado). Essas medidas podem ser usadas para desenvolver modelos empíricos de estimativa do esforço total de projeto, do esforço para criação de página, do esforço de autorização de mídia e do esforço de scripting. No entanto, há ainda mais trabalho a ser feito para que esses modelos possam ser usados com segurança.

FERRAMENTAS DO SOFTWARE

**Estimativa de mão de obra e custo**

Objetivo: o objetivo das ferramentas de estimativa de esforço e custo é fornecer a uma equipe de projeto as estimativas de esforço necessário, duração do projeto e custo, de uma maneira que trate das características específicas do projeto e o ambiente no qual o projeto deve ser criado.

Mecânica: em geral, as ferramentas de estimativa de custo usam uma base de dados histórica derivada de projetos e dados locais coletados na prática, e um modelo empírico (por exemplo, COCOMO II) utilizado para derivar as estimativas de esforço, duração e custo. As características do projeto e o ambiente de desenvolvimento constituem a entrada, e a ferramenta proporciona como saída uma variedade de estimativas.

Ferramentas representativas:¹⁵

Costar, desenvolvida pela Softstar Systems (www.softstarsystems.com), usa o modelo COCOMO II para desenvolver estimativas de software.

Cost Xpert, desenvolvida pelo Cost Xpert Group, Inc. (www.costxpert.com), integra múltiplos modelos de estimativa e uma base de dados históricos de projeto.

Estimate Professional, desenvolvida pelo Software Productivity Centre, Inc. (www.spc.com), é baseada em COCOMO II e no modelo SLIM.

Knowledge Plan, desenvolvida pela Software Productivity Research (www.spr.com), usa entrada de pontos de função como guia principal para um pacote completo de estimativas.

Price S, desenvolvida pela Price Systems (www.pricesystems.com), é uma das ferramentas de estimativa mais antigas e amplamente utilizadas para projetos de desenvolvimento de software de larga escala.

SEER/SEM, desenvolvida pela Galorath, Inc. (www.galorath.com), fornece recursos de estimativa abrangentes, análise de sensibilidade, avaliação de risco e outras características.

SLIM-Estimate, desenvolvida pela QSM (www.qsm.com), faz uso de “conhecimento industrial” abrangente para proporcionar uma “verificação de plausibilidade” para estimativas derivadas usando dados locais.

26.10 A DECISÃO FAZER/COMPRAR

Em grande número de áreas de aplicação de software, muitas vezes é mais viável em termos de custo adquirir em vez de desenvolver software de computador. Os gerentes de engenharia de software precisam tomar a decisão fazer/comprar, que pode ser ainda mais complicada por uma série de opções de aquisição: (1) o software pode ser comprado (ou licenciado) pronto para uso, (2) componentes de software “totalmente testados” ou “parcialmente testados” (veja a Seção 26.4.2) podem ser adquiridos e depois modificados e integrados para atender a necessidades específicas, ou (3) o software pode ser criado de forma personalizada por um terceiro contratado que atenda às especificações do comprador.

As etapas envolvidas na aquisição de software são definidas pelo aspecto crítico do software a ser comprado e custo final. Em alguns casos (por exemplo, software de PC de baixo custo), é menos dispendioso comprar e experimentar do que fazer uma demorada avaliação dos pacotes de software potenciais. Na análise final, a decisão fazer/comprar é tomada com base nas seguintes condições: (1) A data de entrega do software será mais adiantada comparada com o software desenvolvido internamente? (2) O custo da aquisição mais o custo da personalização serão inferiores ao custo para desenvolver o software internamente? (3) O custo do suporte externo (por exemplo, um contrato de manutenção) será menor do que o custo do suporte interno? Essas condições se aplicam a cada uma das opções de aquisição.

26.10.1 Criando uma árvore de decisões

As etapas que acabamos de descrever podem ser ampliadas por meio de técnicas estatísticas como a árvore de análise de decisões.¹⁶ Por exemplo, a Figura 26.8 mostra uma árvore de decisões para um sistema X baseado em software. Nesse caso, a organização de engenharia de software pode (1) criar o sistema X a partir do zero, (2) reutilizar componentes existentes parcialmente testados para criar o sistema, (3) comprar um software disponível e modificá-lo para atender às necessidades locais, ou (4) contratar o desenvolvimento de software externamente.

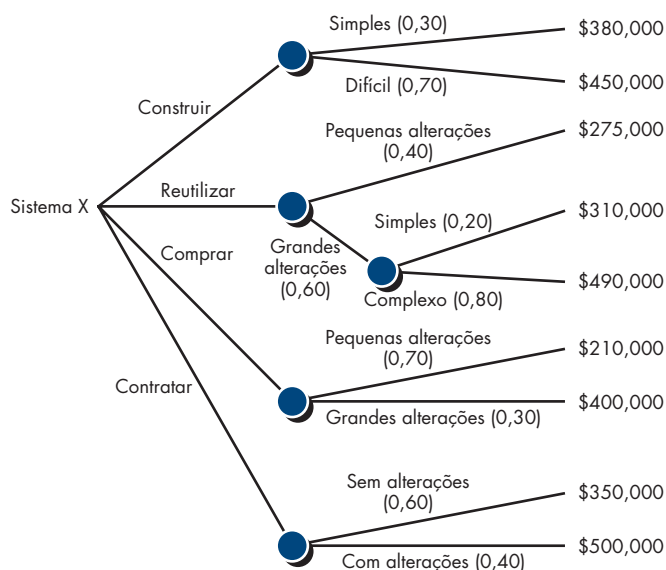
? Existe uma maneira sistemática de escolher entre as opções associadas com a decisão fazer/comprar?

¹⁵ As ferramentas aqui apresentadas não significam um aval, mas sim uma amostra dessa categoria. Na maioria dos casos, seus nomes são marcas registradas pelos respectivos desenvolvedores.

¹⁶ Uma boa introdução à árvore de análise de decisões pode ser encontrada no endereço http://en.wikipedia.org/wiki/Decision_tree.

FIGURA 26.8

Uma árvore de decisões para apoiar a decisão fazer/comprar



Se o sistema tem de ser criado desde o início, há uma probabilidade de 70% de que a tarefa será difícil. Usando as técnicas de estimativa discutidas neste capítulo, o planejador do projeto estima que um trabalho de desenvolvimento difícil custará \$ 450 mil. Um trabalho de desenvolvimento “simples” tem um custo estimado de \$ 380 mil. O valor esperado do custo, calculado ao longo de qualquer ramo da árvore de decisões, é

$$\text{Custo esperado} = \sum (\text{probabilidade do caminho})_i \times (\text{custo estimado do caminho})_i$$

em que i é o caminho da árvore de decisões. Para o caminho da criação,

$$\text{Custo esperado}_{\text{construção}} = 0,30 (\$ 380\text{K}) + 0,70 (\$ 450\text{K}) = \$ 429\text{K}$$

Seguindo outros caminhos da árvore de decisões, são mostrados os custos projetados para reutilização, compra e contratação, sob várias circunstâncias. Os custos esperados para esses caminhos são

$$\text{Custo esperado}_{\text{reutilizar}} = 0,40 (\$ 275\text{K}) + 0,60 [0,20 (\$ 310\text{K}) + 0,80 (\$ 490\text{K})] = \$ 382\text{K}$$

$$\text{Custo esperado}_{\text{comprar}} = 0,70 (\$ 210\text{K}) + 0,30 (\$ 400\text{K}) = \$ 267\text{K}$$

$$\text{Custo esperado}_{\text{contratar}} = 0,60 (\$ 350\text{K}) + 0,40 (\$ 500\text{K}) = \$ 410\text{K}$$

Com base na probabilidade e nos custos projetados da Figura 26.8, o custo esperado mais baixo é o da opção “comprar”.

No entanto, é importante observar que muitos outros critérios – não apenas o custo – devem ser considerados durante o processo de tomada de decisões. A disponibilidade, a experiência do desenvolvedor/fornecedor/contratado, a conformidade com os requisitos, as “políticas” locais e a possibilidade de alterações são apenas alguns dos critérios que podem afetar a decisão final de construir, reutilizar, comprar ou contratar.

26.10.2 Terceirização

Mais cedo ou mais tarde, toda empresa que desenvolve software faz a pergunta fundamental: “Há alguma maneira de obtermos o software e sistemas que precisamos a um preço mais baixo?”. A resposta não é nada simples, e as discussões emocionais que ocorrem sempre levam a uma única palavra: *terceirização*.

Conceitualmente, a terceirização é extremamente simples. As atividades de engenharia de software são contratadas de um terceiro que faz o trabalho a um custo menor e, espera-se, com

melhor qualidade. O trabalho de software realizado em uma empresa é reduzido a uma atividade de gerenciamento de contrato.¹⁷

A decisão de optar pela *terceirização* pode ser estratégica ou tática. No nível estratégico, os gerentes de negócio consideram se uma parte significativa de todo o trabalho de software pode ser contratada com outros. No nível tático, um gerente de projeto determina se parte ou todo o projeto pode ser mais bem executado subcontratando o trabalho de software.

Independentemente da amplitude do foco, a decisão de terceirizar muitas vezes é financeira. Uma discussão detalhada da análise financeira para a terceirização está além do escopo deste livro; aconselhamos a consulta de outros autores (por exemplo, [Min95]). No entanto, apresentaremos aqui um rápido exame dos prós e contras dessa decisão.

No lado positivo, podem ser obtidas reduções de custos diminuindo-se o número de pessoas empregadas e instalações (por exemplo, computadores, infraestrutura) que os suportam. No lado negativo, a empresa perde algum controle sobre o software de que precisa. Como software é uma tecnologia que diferencia seus sistemas, serviços e produtos, uma empresa corre o risco de colocar o destino de sua competitividade nas mãos de um terceiro.

A tendência para a terceirização certamente continuará. A única maneira de atenuá-la é reconhecer que o trabalho de software é extremamente competitivo em todos os níveis. A única maneira de sobreviver é se tornar tão competitivo quanto os próprios fornecedores de terceirização.

“Como regra, a terceirização requer uma habilidade de gerenciamento ainda maior do que o desenvolvimento interno.”

Steve McConnell

CASA SEGURA



Terceirização

Cena: Sala de reuniões da CPI Corporation no início do projeto.

Atores: Mal Golden, gerente sênior do setor de desenvolvimento de produto; Lee Warren, gerente de engenharia; Joe Camalleri, vice-presidente executivo do setor de desenvolvimento de negócio; e Doug Miller, gerente de projeto de engenharia de software.

Conversa:

Joe: Estamos considerando a terceirização da parte de engenharia de software do *CasaSegura*.

Doug (chocado): Quando isso ocorreu?

Lee: Recebemos uma cotação de um desenvolvedor do exterior. Ela está 30% abaixo daquilo que o seu grupo acha que custará. Veja. [Passa a cotação para Doug, que a lê.]

Mal: Como você sabe, Doug, estamos tentando baixar os custos em 30%, e 30% são 30%. Além disso, esse pessoal é bastante recomendado.

Doug (tomando fôlego e tentando manter-se calmo): Bem, vocês me pegaram de surpresa. Antes de tomarmos a decisão final, gostaria de fazer alguns comentários.

Joe (concordando): Claro, prossiga.

Doug: Nunca trabalhamos com essa empresa de terceirização antes, certo?

Mal: Certo, mas...

Doug: E eles afirmam que quaisquer alterações nas especificações serão cobradas a uma taxa adicional, certo?

Joe (contrariado): Certo, mas esperamos que as coisas se mantenham razoavelmente estáveis.

Doug: Uma péssima suposição, Joe.

Joe: Bem...

Doug: É provável que iremos lançar novas versões deste produto durante os próximos anos. E é razoável supor que o software trará muitas das novas características, certo? [Todos acenam afirmativamente.]

Doug: Nós já coordenamos um projeto internacional antes?

Lee (parecendo preocupado): Não, mas me disseram...

Doug (tentando conter sua raiva): Então o que você está me dizendo é que: (1) vamos trabalhar com um fornecedor desconhecido, (2) os custos para fazer isso não são tão baixos quanto parecem, (3) estamos de fato concordando em trabalhar com eles durante muitas versões do produto, não importa o que fizerem na primeira versão, e (4) vamos aprender sobre um projeto internacional “com o bonde andando”. [Todos permanecem em silêncio.]

Doug: Pessoal... Eu acho que isso é um erro, e gostaria que vocês reconsiderassem por mais um dia. Teremos muito mais controle se fizermos o trabalho internamente. Temos a experiência, e posso garantir que não vai nos custar muito mais... O risco será menor e sei que todos vocês têm aversão a riscos, como eu também tenho.

Joe (contrariado): Você apresentou boas razões, mas tem um interesse claro em manter esse projeto internamente.

Doug: Isso é verdade, mas isso não muda os fatos.

Joe (com um suspiro): Ok, vamos adiar por um ou dois dias, pensar um pouco e nos reunimos novamente para uma decisão final. Doug, posso falar em particular com você?

Doug: Claro... Eu realmente quero ter certeza de que estamos fazendo a coisa certa.

17 A terceirização pode ser vista em geral como qualquer atividade que leva à aquisição de software ou componentes de software de uma fonte fora da organização de engenharia de software.

26.11 RESUMO

Um planejador de projeto de software deve estimar três itens antes de começar: quanto tempo levará, quanto esforço será necessário e quantas pessoas serão envolvidas. Além disso, o planejador deve prever os recursos (hardware e software) que serão necessários e o risco envolvido.

A definição de escopo ajuda o planejador a desenvolver estimativas usando uma ou mais técnicas que se dividem em duas grandes categorias: decomposição e modelagem empírica. As técnicas de decomposição requerem um delineamento das principais funções do software, seguido de estimativas de (1) número de LOC, (2) valores selecionados dentro do domínio de informações, (3) número de casos de uso, (4) número de pessoas-mês necessário para implementar cada função, ou (5) número de pessoas-mês necessário para cada atividade de engenharia de software. Técnicas empíricas usam expressões derivadas empiricamente para esforço e tempo para prever esses valores de projeto. Podem ser utilizadas ferramentas automáticas para implementar um modelo empírico específico.

Estimativas precisas de projeto em geral usam pelo menos duas das três técnicas que acabamos de descrever. Comparando e reconciliando as estimativas desenvolvidas por meio de diferentes técnicas, o planejador tem maior possibilidade de derivar uma estimativa precisa. A estimativa de projeto de software nunca pode ser uma ciência exata, mas uma combinação de bons dados históricos e técnicas sistemáticas pode melhorar a precisão da estimativa.

PROBLEMAS E PONTOS A PONDERAR

26.1. Suponha que você seja o gerente de projeto de uma empresa que cria software para robôs de uso doméstico. Você foi contratado para criar o software para um robô que corta a grama de uma residência. Apresente uma definição de escopo que descreva o software. Certifique-se de que a sua definição de escopo esteja delimitada. Se não estiver familiarizado com robôs, faça algumas pesquisas antes de começar a escrever. Além disso, formule suas hipóteses sobre o hardware necessário. Alternativa: Troque o robô cortador de grama por um outro problema de seu interesse.

26.2. A complexidade do projeto de software é discutida rapidamente na Seção 26.1. Desenvolva uma lista de características de software (por exemplo, operação concorrente, saída gráfica) que afetam a complexidade de um projeto. Defina prioridades na lista.

26.3. O desempenho é uma consideração importante durante o planejamento. Discuta como pode ser interpretado diferentemente, dependendo da área de aplicação do software.

26.4. Faça uma decomposição funcional do software do robô que você descreveu no Problema 26.1. Estime o tamanho de cada função em LOC. Supondo que a sua organização produza 450 LOC/pm com um valor bruto de mão de obra de \$ 7 mil por pessoa-mês, estime a mão de obra e custo necessário para criar o software usando a técnica de estimativa baseada em LOC descrita neste capítulo.

26.5. Use o modelo COCOMO II para estimar o esforço necessário para criar software para um caixa eletrônico simples que produz 12 telas, 10 relatórios e requer aproximadamente 80 componentes de software. Suponha que a complexidade seja média e que a maturidade desenvolvedor/ambiente seja média. Use o modelo de composição de aplicação com pontos de objetos.

26.6. Use a equação do software para estimar o software do robô cortador de grama. Assuma que a Equação (26.4) seja aplicável e que $P = 8.000$.

26.7. Compare as estimativas de esforço derivadas nos Problemas 26.4 e 26.6. Qual o desvio-padrão, e como ele afeta o seu grau de certeza sobre a estimativa?

26.8. Com os resultados obtidos no Problema 26.7, determine se é razoável esperar que o software possa ser criado nos próximos seis meses e quantas pessoas serão empregadas para isso.

26.9. Desenvolva um modelo de planilha que implemente uma ou mais das técnicas de estimativa descritas neste capítulo. Como alternativa, use um ou mais modelos on-line para estimativa de fontes da Web.

26.10. Para uma equipe de projeto: desenvolver uma ferramenta de software que implemente cada uma das técnicas de estimativa desenvolvidas neste capítulo.

26.11. Parece estranho que as estimativas de custo e de cronograma sejam feitas durante o planejamento do projeto de software – antes de fazer a análise detalhada dos requisitos de software ou projeto. Por que você acha que isso é feito? Há circunstâncias nas quais não deveria ser feito?

26.12. Calcule novamente os valores esperados para a árvore de decisões da Figura 26.8 supondo que cada ramo tem uma probabilidade 50-50. Como isso muda a sua decisão final?

LEITURAS E FONTES DE INFORMAÇÃO COMPLEMENTARES

Muitos livros de gerenciamento de projeto de software contêm discussões de estimativas de projeto. O Project Management Institute (*PMBOK Guide*, PMI, 2001), Wysoki e seus colegas (*Effective Project Management*, Wiley, 2000), Lewis (*Project Planning Scheduling and Control*, 3d ed., McGraw-Hill, 2000), Bennatan (*On Time, Within Budget: Software Project Management Practices and Techniques*, 3d ed., Wiley, 2000), e Phillips [Phi98] relacionam informações úteis sobre estimativas.

McConnell (*Software Estimation: Demystifying the Black Art*, Microsoft Press, 2006) escreveu um guia prático que fornece amplas diretrizes para qualquer um que precise estimar custo de software. Parthasarathy (*Practical Software Estimation*, Addison-Wesley, 2007) destaca pontos de função como uma métrica de estimativa. Laird e Brennan (*Software Measurement and Estimation: A Practical Approach*, Wiley-IEEE Computer Society Press, 2006) tratam de medida e seu uso na estimativa de software. Pfleeger (*Software Cost Estimation and Sizing Methods, Issues, and Guidelines*, RAND Corporation, 2005) desenvolveu um guia resumido que trata de muitos fundamentos de estimativas. Jones (*Estimating Software Costs*, 2d ed., McGraw-Hill, 2007) escreveu um dos mais abrangentes tratamentos de modelos e dados aplicáveis a estimativas de software em todos os domínios de aplicação. Coombs (*IT Project Estimation*, Cambridge University Press, 2002) e Roetzheim e Beasley (*Software Project Cost and Schedule Estimating: Best Practices*, Prentice-Hall, 1997) apresentam muitos modelos úteis e sugerem diretrizes passo a passo para gerar as melhores estimativas possíveis.

Há disponível na Internet uma ampla variedade de fontes de informação sobre estimativa de software. Pode-se encontrar uma lista atualizada de referências da Web no site **www.mhhe.com/engcs/compsci/pressman/professional/olc/ser.htm**.